

Министерство образования и науки
Краснодарского края
ГБПОУ КК ЕПК

Отделение педагогики
Специальность 09.02.07 Информационные системы и программирование

Студент И-42 группы
САМСЫКА РОДИОН МИХАЙЛОВИЧ

**РАЗРАБОТКА ПРОГРАММЫ ДЛЯ АВТОМАТИЗАЦИИ ОФОРМЛЕНИЯ
ЗАКАЗОВ В РЕСТОРАНЕ**

Дипломная работа (Дипломный проект)

Научный руководитель:
Фомин Андрей Томасович

г. Ейск, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	4
1 Модель предметной области.....	4
2 Предварительное определение требований.....	8
2.1 Общее описание.....	8
2.2 Функциональные требования.....	10
2.3 Требования к данным.....	10
2.4 Требования к внешним интерфейсам.....	12
2.5 Атрибуты качества.....	13
3 Показатели экономической эффективности.....	14
3.1 Расчет себестоимости программного продукта.....	14
1.3.2 Расчет экономического эффекта программного продукта.....	15
ГЛАВА 2 РЕАЛИЗАЦИЯ ПРОГРАММНЫХ МОДУЛЕЙ.....	18
1 Технологии программирования.....	18
1.1 Методы разработки.....	18
2.2 Описание программных модулей.....	19
2.3 Интеграция программных модулей.....	19
2.4 Описание программного интерфейса.....	20
2.4.1 Первый пункт.....	20
2.4.2 Это второй пункт.....	20
ЗАКЛЮЧЕНИЕ.....	22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	23
ПРИЛОЖЕНИЕ А.....	24
ПРИЛОЖЕНИЕ Б.....	26
ПРИЛОЖЕНИЕ В.....	27

ВВЕДЕНИЕ

Тема моего проекта «Разработка программы для автоматизации оформления заказов в ресторане».

Предмет – автоматизация информационных процессов предметной области «Ресторан».

Цель работы – разработка автоматизированной системы для оформления заказов в предметной области «Ресторан».

Объект – предметная область «Ресторан».

Для достижения поставленной цели курсового проекта необходимо решить следующие задачи:

- 1) проанализировать предметную область и обосновать актуальность проекта.
- 2) выбрать язык программирования и среду разработки;
- 3) спроектировать структуру и модули программного обеспечения;
- 4) разработать программное обеспечение.

Структура работы: курсовой проект состоит из введения, двух глав, заключения, списка литературы. Во введении обоснована актуальность темы исследования. В первой главе описано проектирование системы. Вторая глава посвящена описанию процесса разработки программных модулей.

ГЛАВА 1 ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

1 Модель предметной области

Ресторан — это предприятие общественного питания [2, с.3], предоставляющее услуги в сфере кулинарии повышенной категории. Он специализируется на производстве, реализации и организации потребления кулинарной продукции, включая сложные, фирменные блюда и напитки. Основным способом организации потребления является подача блюд в зале ресторана [1, с. 1].

В отличие от заведений более простого формата, ресторан делает акцент на уникальном меню, высоком уровне сервиса и создании особой атмосферы, что формирует его ценностное предложение для гостей. Несмотря на более высокий ценовой сегмент, ресторан, как и любое предприятие общественного питания, является важной и высококонкурентной сферой деятельности, где борьба за клиента ведется не только за счет качества кухни, но и за счет общего впечатления и уровня обслуживания.

Предварительное моделирование предметной области позволяет выявить основные бизнес-процессы. К ним относятся приём и обработка заказов от гостей, передача заказов на кухню и в бар, непосредственно процесс приготовления блюд и напитков, обслуживание клиентов, а также учёт и управление складскими запасами. Иногда в ресторанах сохраняются устаревшие, не автоматизированные методы работы: официанты принимают заказы, записывая их в блокноты или на листы бумаги, которые затем вручную переносятся на кухню и в бар. Для обеспечения бесперебойного процесса приготовления необходимы поставки продуктов, что требует постоянного контроля за их наличием на складе, проведения инвентаризации и своевременного формирования заявок для поставщиков. Отсутствие единой автоматизированной системы для управления этими процессами приводит к

риску человеческих ошибок, замедлению обслуживания и сложностям в оперативном управлении ресурсами.

Процесс приема заказа в ресторане представляет собой последовательность взаимосвязанных операций, начинающихся с установления контакта с гостем и заканчивающихся передачей сформированного заказа на производство. Изначально официант осуществляет подход к столику с целью приветствия и представления меню, после чего следует этап консультирования по ассортименту блюд и напитков, их особенностям, составу и возможным модификациям, учитывая индивидуальные предпочтения и ограничения гостя, такие как аллергенные компоненты или диетические требования. На этапе фиксации заказа официант детально записывает выбранные позиции на бумажный бланк заказа с указанием точных наименований, количества порций, специфических пожеланий по приготовлению и подаче, включая степень прожарки для мясных блюд или исключение определенных ингредиентов из салатов. После полного формирования перечня заказанных позиций официант повторно озвучивает гостю весь список для окончательного подтверждения и устранения возможных недоразумений, после чего заказ считается принятым и подлежит дальнейшей обработке. Завершающей стадией процесса является передача оформленного заказа на кухню и в бар, где он регистрируется, получает уникальный идентификатор и поступает в работу к соответствующим поварам и барменам, при этом официант отмечает время приема заказа для последующего контроля сроков его выполнения. На рисунке 1 изображена схема работы с заказом на текущий момент.

Процесс управления складом в ресторане представляет собой непрерывный цикл взаимосвязанных операций, начинающихся с планирования потребности в продукции и заканчивающихся списанием использованных ингредиентов. Первоначальным этапом является составление заявки на поставку продуктов на основе анализа текущих остатков и прогнозируемого объема производства, при этом учитываются такие факторы, как сезонность,

планируемые акции и исторические данные о расходе. После формирования заявки осуществляется процедура приемки поставки, которая включает проверку соответствия количества и качества прибывшей продукции товарной накладной, а также контроль соблюдения температурных режимов и сроков годности. Принятые товары перемещаются в соответствующие зоны хранения с соблюдением принципа товарного соседства и маркировкой даты поступления для обеспечения ротации запасов. В процессе хранения регулярно проводится инвентаризация остатков с фиксацией фактического наличия продуктов и выявлением расхождений с данными системы.

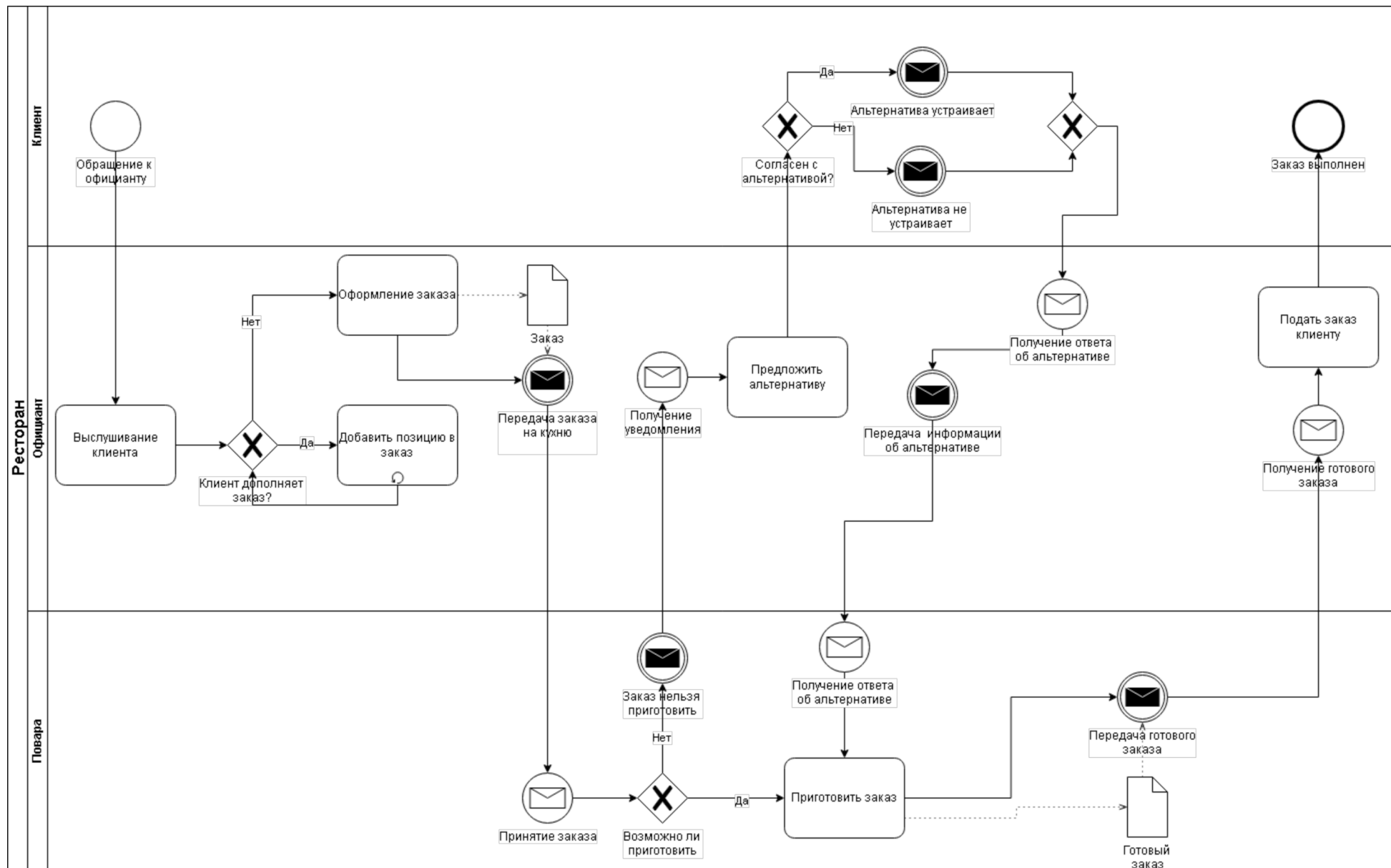


Рисунок 1 — Схема работы с заказом на данный момент

2 Предварительное определение требований

2.1 Общее описание

Разрабатываемое приложение предназначено для автоматизации ключевых бизнес-процессов ресторана. Основные функции включают приём и обработку заказов, управление складскими запасами в реальном времени и формирование финансовых отчётов [5]. Система строится на архитектуре с использованием Qt Quick для клиентских приложений на планшетах и TCP-соединения для взаимодействия между устройствами в локальной сети.

2.1.1 Общий взгляд на продукт

Приложение помогает официантам в оформлении заказов, предоставляя функции для более удобной записи. Также оно помогает отслеживать продукт на складе. Приложение новое, создаётся с нуля. На рисунке 2 изображена схема работы с заказом после внедрения программного продукта.

2.1.2 Классы и характеристики пользователей

Приложением пользуются официанты, администрация, кладовщики. Официанты оформляют заказы; кладовщики обновляют информацию о продукте на складе; администрация проверяет приложение на соответствие данных, работоспособность.

2.1.3 Ограничение дизайна и реализации

Приложение создаётся с нуля, что позволяет выбрать средства разработки на своё усмотрение. Так как приложение создаётся для планшетов, следует создать крупный интерфейс и использовать кроссплатформенную разработку, такую как Qt quick.

2.1.4 Предположения и зависимости

Для функционирования системы требуется корректно работающая локальная сеть. Установка дополнительных программ или приложений не требуется.

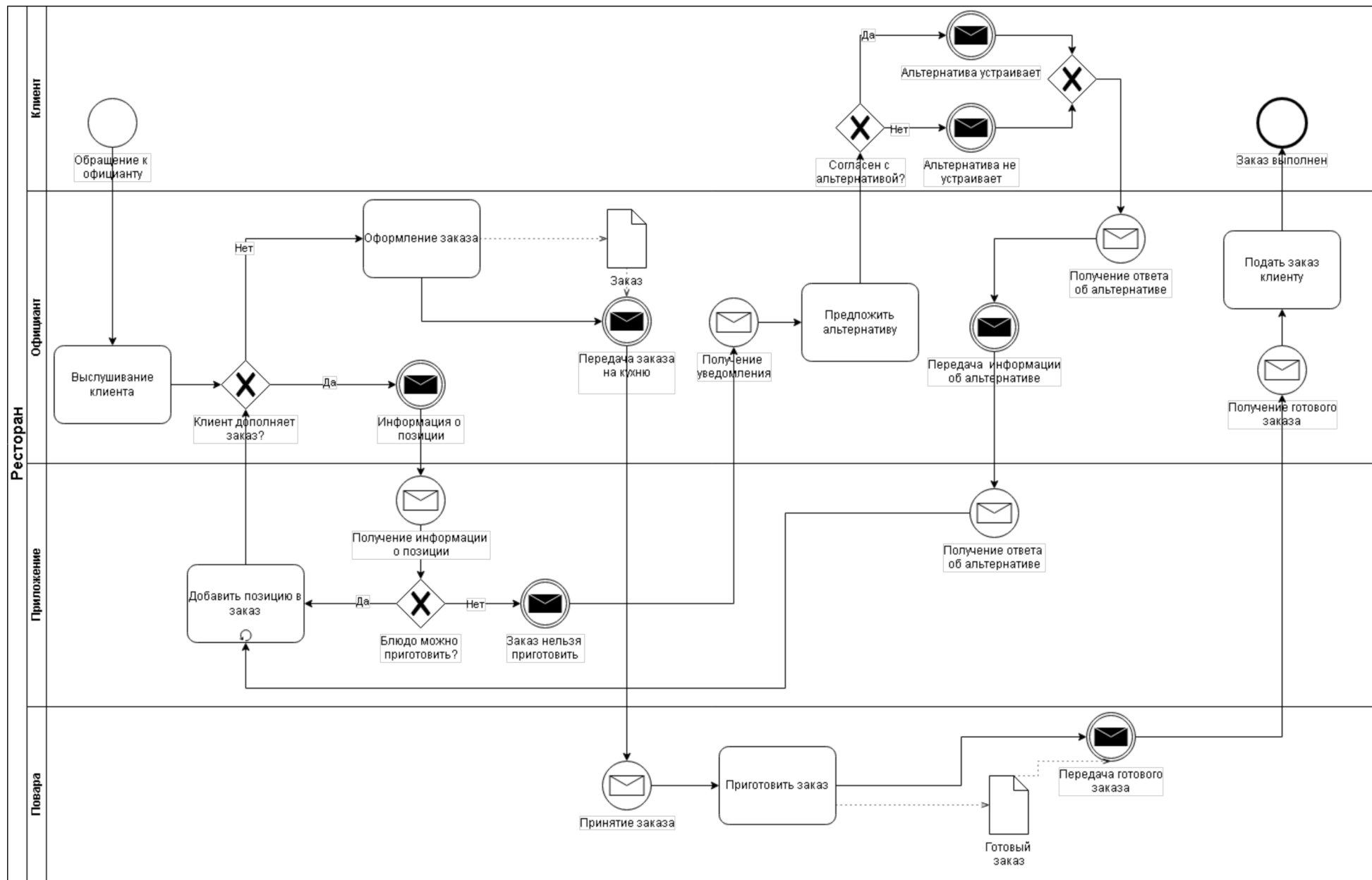


Рисунок 2 — Схема работы с заказом при внедрении программного продукта

2.2 Функциональные требования

FUNC-001. Синхронизация: при запуске приложения загружаются актуальные данные. Все изменения должны синхронизироваться с имеющимися данными при нажатии кнопки обновления данных.

FUNC-002. Оформление заказов: создание нового заказа в базе данных с привязкой официанта (берётся из логина) и столика (вводится вручную) к нему. [4, с. 108]

FUNC-003. Фильтрация меню на окне оформления заказа по категории и доступности.

FUNC-004. Добавление блюда в заказ: официант нажимает на блюдо в списке, вводит количество и нажимает «Добавить». Блюдо появляется в текущем заказе.

FUNC-005. Автоматическое списание: при добавлении блюда в заказ происходит списание продуктов на складе. Если удалить блюдо из заказа, то продукты вернутся.

FUNC-006. Предупреждение о малом остатке: предупреждение о малом количестве продуктов для блюда перед его добавлением в заказ. При этом добавить блюдо можно.

FUNC-007. Проверка перед добавлением: если продуктов не хватает, то добавить в заказ блюдо нельзя. [4, с. 108]

FUNC-008. Изменение количества на складе: возможность кладовщику изменять количество продукта на складе, как добавлением/отниманием, так и установлением.

2.3 Требования к данным

2.3.1 Логическая модель данных

DATA-001. Основные сущности которые отразятся в базе данных:

- Пользователь — пользователи приложения, оформляет заказ или редактирует склад;

- Заказ — то, что записывается в базу данных. Хранит информацию о заказанных блюдах;
- Заказанное блюдо — блюдо которое запишется в заказ, его количество и просьба от клиента;
- Блюдо — экземпляр блюда который указан в меню, ссылается на состав;
- Продукт в составе — продукт который входит в состав блюда;
- Продукт — продукт для приготовления блюда.

На рисунке 3 представлена логическая модель данных, описывающая основные сущности в выбранном бизнес процессе.

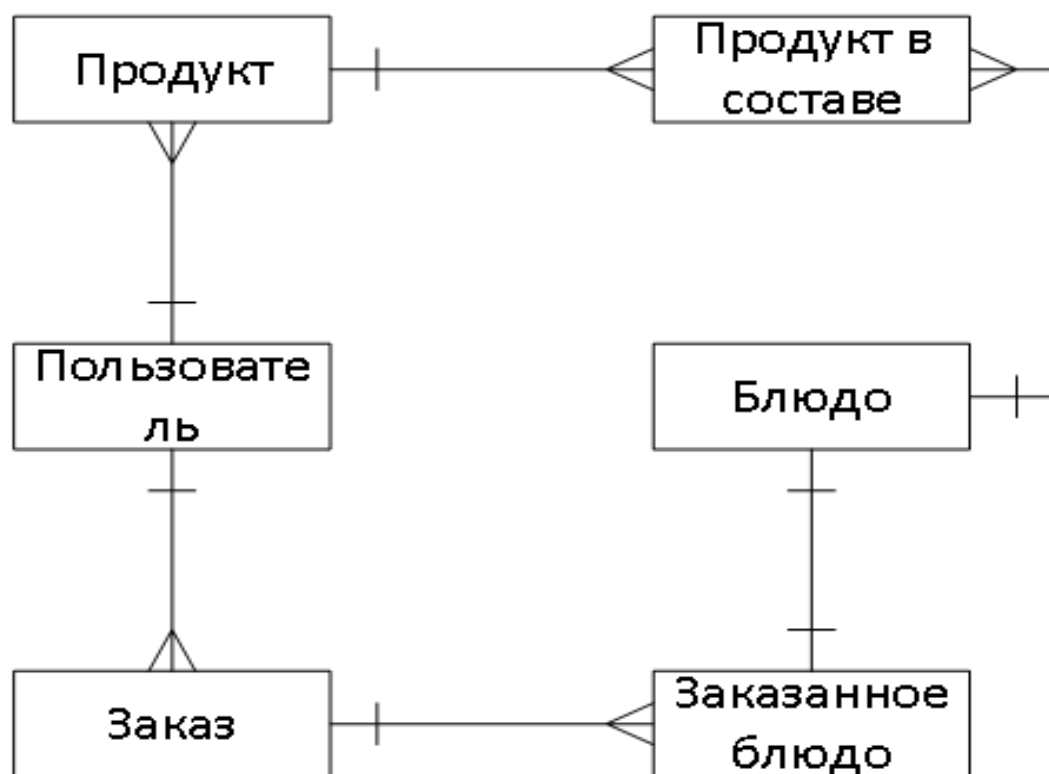


Рисунок 3 — Модель данных

2.3.2 Отчёты

DATA-002. Отчёт о выручке: формирование отчёта о средней выручке с учётом блюд, столиков, официанта и даты.

DATA-003. Список о малом остатке: формирование списка продуктов, которых осталось меньше указанного значения.

DATA-004. Популярность блюд: формирование списка блюд по тому, как часто их заказывают.

2.3.3 Получение, целостность, хранение и утилизация данных

Данные поступают из следующих источников:

- Планшеты официантов: данные о заказах в реальном времени.
- Планшеты кладовщиков: данные об остатках и операциях с ингредиентами.

DATA-005. Все данные хранятся централизованно в базе данных на устройстве-сервере, обеспечивая единый источник информации для синхронизации.

2.4 Требования к внешним интерфейсам

2.4.1 Пользовательские интерфейсы

Основные требования:

- Отсутствие отвлекающих элементов.
- Использование крупного шрифта и четкой цветовой гаммы.
- Минимизация текста для быстрого восприятия информации.

UI-001. Меню: можно открыть меню навигации, содержащее кнопки для перехода на каждую из форм, внизу располагается кнопка для смены учётной записи. Сверху ФИО пользователя. Кнопки выводятся в соответствии с ролью текущего пользователя.

UI-002. Оформление заказа: разделение на области меню, ввод данных и кнопки, информация о текущем заказе. Выпадающие списки для выбора фильтра. Поле для ввода количества и номера столика, дополнительных требований, кнопки «Добавить к заказу», «Удалить из заказа» и «Оформить заказ».

UI-003. Работа со складом: вывод всех продуктов с возможностью редактировать остатки. Поле для ввода числа и кнопки «Добавить», «Убавить» и «Установить».

UI-004. Отчёты: выбор отчёта из списка (DATA-003, DATA-004), поля для ввода фильтров для каждого из отчётов, область с выводом данных выбранного отчёта.

UI-005. Настройки: окно настроек базы данных, для сервера; сетевые настройки для соединения с сервером; прочие настройки для дополнительных функций.

2.4.2 Интерфейсы ПО

Система работает на базе операционной системы Android версии не ниже 9. Для взаимодействия с базой данных используется СУБД PostgreSQL с использованием языка SQL, так как он обеспечивает корректную работу в системе «Клиент-Сервер» [7, с. 34].

2.4.3 Интерфейсы оборудования

Планшеты связываются между собой по локальной сети, обеспечивая обмен данными при обновлении.

2.5 Атрибуты качества

2.5.1 Удобство использования

NF-001. Для добавления блюда в заказ требуется не более 3 нажатий (выбрать блюдо — ввести количество — «Добавить»).

NF-002. Для изменения остатка на складе требуется не более 3 нажатий (выбрать продукт — ввести количество — «Добавить»/«Убавить»/«Установить»).

2.5.2 Производительность

PERF-001. Обновление данных занимает не более 3 секунд.

PERF-002. Отзывчивый интерфейс с задержкой выполнения операций не более 1 секунды.

2.5.3 Безопасность

SEC-001. Авторизация происходит по уникальному логину и паролю.

SEC-002. Разные роли видят разные пункты меню. Официант не перейдёт на склад, а кладовщик не перейдёт на окно оформления заказа.

2.5.4 Техника безопасности

SEC-003. Хранение данных в надёжных местах и не передавать иным лицам.

3 Показатели экономической эффективности

3.1 Расчет себестоимости программного продукта

Основной задачей технико-экономического обоснования является определение технического и экономического эффекта от его использования.

Выполним расчет себестоимости программного продукта по формуле 1.

$$C = M_{вс} + Э + З_{зн}^o + З_{зн}^д + З_{сн} + Н \quad (1)$$

где $M_{вс}$ — затраты на вспомогательные материалы, **Р**.

$Э$ — затраты на электроэнергию и технологические цели, **Р**

$З_{зн}^o$ — основная зарплата работника, **Р**

$З_{зн}^д$ — дополнительная зарплата разработчика, **Р**

$З_{сн}$ — отчисления на соц. нужды, **Р**

$Н$ — накладные расходы, **Р**

Можно произвести расчёты для каждой переменной.

Вспомогательные материалы:

Из таблицы 1 видно, что вспомогательные материалы в сумме стоят 300 р.

Затраты на электроэнергию:

180 дней по 8 часов.

$$Э = 0.5 * 7.5 * 1\,440 * 0.6$$

$$Э = 3\,240$$

Общая зарплата:

270 000 р.

Дополнительная зарплата:

27 000 р.

Отчисления на социальные нужды:

$(270000 + 27000) \cdot 0.3 = 89\,100$ р.

Накладные расходы:

$(270000 + 27000) \cdot 0.1 = 29\,700$ р.

Таким образом: $C = 300 + 3\,240 + 270000 + 27000 + 89\,100 + 29\,700$

$C = 419\,340$ р.

Таблица 1 — Затраты на вспомогательные материалы

	Наименование затрат	Кол-во, шт.	Сумма, ₽.
	Память USB FLASH DRIVE 2Gb	1	200
	Переходник USB-A USB-C	1	100
	Итого		300

1.3.2 Расчет экономического эффекта программного продукта

Экономический эффект от внедрения ПО рассчитывается по формуле 2

$$\mathcal{E}_{\text{эф}} = N \cdot (t_1 - t_2) \cdot C_{\text{чтс}} \cdot K_{\text{доп}} \cdot K_{\text{сн}} - E_n \cdot K \quad (2)$$

где

N — количество обрабатываемых документов (взять $N = 6000$);

t_1, t_2 — трудоемкость обработки документов до и после внедрения программы:

$t_1 = 15, t_2 = 7$

$C_{\text{чтс}}$ — часовая тарифная ставка пользователя, р./час;

$K_{\text{доп}}$ — коэффициент отчислений на дополнительную заработную плату,
 $K_{\text{доп}} = 0,1$;

$K_{\text{сн}}$ — коэффициент отчислений на соц. нужды, $K_{\text{сн}} = 0,3$;

E_n — нормативный коэффициент окупаемости капитальных вложений, $E_n = 0,39$;

K — доп. капиталовложения, связанные с разработкой программного продукта (здесь полная себестоимость рассчитанная выше).

Используя ранее рассчитанные значения и подставляя в формулу выше, можно выяснить что: $\Delta = 6000 * (15 - 7) * 165 * 0.1 * 0.3 - 0.39 * 419\,340$

Срок окупаемости программного продукта, рассчитанный по формуле 3: где C_{np} — себестоимость программного продукта, ₽

C_1 — стоимостные затраты при использовании ручного труда, ₽

C_2 — стоимостные затраты при использовании программы, ₽

$419340 / (281\,957,4 - 44\,357,4) = 1.8$ лет

Ручной труд:

281 957,4

Использование программы:

44 357,4

$$T_{ок} = \frac{C_{np}}{C_1 - C_2} \quad (3)$$

В таблице 2 выведены результаты произведённых вычислений.

Таблица 2 — Итоговые значения показателей экономической эффективности

Наименование показателя	Ед. Изм.	Затраты, р.
1. Основная заработная плата разработчика	р.	270 000
2. Дополнительная заработная плата разработчика	р.	27 000
3. Отчисления на социальные нужды	р.	89 100
4. Накладные расходы	р.	29 700
5. Затраты на электроэнергию	р.	3 240
6. Вспомогательные материалы	р.	300

7. Полная себестоимость программного продукта	р.	419 340
8. Экономический эффект	р.	74 057,4
9. Срок окупаемости программного продукта	лет	1,8

ГЛАВА 2 РЕАЛИЗАЦИЯ ПРОГРАММНЫХ МОДУЛЕЙ

1 Технологии программирования

Разработка программного продукта потребовала тщательного выбора технологий и методов программирования для достижения поставленных целей. Ниже перечислены используемые технологии и инструменты, их характеристики и обоснование выбора.

1.1 Методы разработки

При выборе методов разработки необходимо учитывать следующие критерии:

- Четкое определение требований на начальном этапе;
- Структурированное планирование работ;
- Последовательное выполнение этапов проекта.

С учетом данных критериев предпочтение отдается водопадной методологии. Она предполагает чёткое и последовательное разделение этапов разработки:

1. Определение требований
2. Проектирование
3. Кодирование
4. Тестирование
5. Эксплуатация

В связи с этим данная методология является наиболее подходящей.

1.2 Языки программирования

В качестве языка программирования для разработки выбран C++. Это обусловлено его высокой производительностью и кроссплатформенностью. C++ используется в Qt Creator для написания функциональной части как десктопных так и для мобильных приложений.

Для взаимодействия с базами данных используется структурированный язык запросов SQL, который характеризуется простотой и многофункциональностью, обеспечивая эффективное управление данными.

1.3 Инструменты и среды разработки

В качестве инструмента для разработки выбрана интегрированная среда разработки Qt Creator, которая предоставляет возможности для создания кроссплатформенных приложений. Для конфигурации и сборки проекта используется система сборки CMake.

Процесс разработки основан на объектно-ориентированном подходе, где каждый компонент программы представляется в виде отдельного объекта или класса. Этот подход соответствует требованиям и спецификациям разрабатываемой программы.

1.4 Архитектурные подходы

Так как приложение является сетевым, то при разработке используется клиент-серверная архитектура. Это предоставит возможность одному устройству работать в режиме сервера, а остальным в режиме клиента.

1.5 Библиотеки и фреймворки

В проекте используются следующие библиотеки и фреймворки:

- Qt 6 – основной фреймворк, предоставляющий классы для работы с графическим интерфейсом (Qt Quick), сетевыми сокетами (QTcpSocket, QTcpServer), базами данных (QSqlDatabase), а также метаобъектную систему (сигналы/слоты, Q_PROPERTY, Q_INVOKABLE).
- Qt SQL – модуль для доступа к базам данных.
- Qt Network – модуль для реализации TCP-клиент-серверного взаимодействия.

1.6 Тестирование и отладка

Для тестирования работоспособности приложения используется класс Logs. Его методы записывают сообщения в лог-файл с разным уровнем важности, а так же дублирует эти сообщения в терминал.

2 Описание программного кода продукта

2.1 Общая архитектура

Программа построена по клиент-серверной архитектуре. В зависимости от настроек приложение может работать в одном из двух режимов:

- Серверный режим – приложение напрямую взаимодействует с СУБД PostgreSQL, принимает TCP-подключения от клиентов и обрабатывает их запросы.
- Клиентский режим – приложение подключается к удалённому серверу через TCP-сокеты и отправляет JSON-запросы, получая ответы.

Взаимодействие с базой данных реализовано в классе DBWorker (сервер) или TcpClient (клиент). Это позволяет QML-интерфейсу не зависеть от режима работы – все методы вызываются одинаково. Основные модули:

- DBWorker – работа с PostgreSQL (только в серверном режиме).
- TcpServer + ClientConnection – TCP-сервер, принимающий запросы и использующий DBWorker.
- TcpClient – TCP-клиент, реализующий тот же API, что и DBWorker, но через сеть.
- Logs – логирование.
- Settings – хранение и загрузка настроек с помощью QSettings.
- QML-файлы – пользовательский интерфейс.

2.2 Модуль работы с базой данных

Класс DBWorker предназначен для выполнения всех операций с базой данных PostgreSQL. Он открывает соединение в конструкторе (параметры подключения жёстко заданы, но могут быть изменены через вызов reconnect).

Основные методы:

- checkUser(login, password) – авторизация пользователя. Из таблицы Users извлекаются соль и хэш пароля (SHA3-256), сравниваются с введенными.
- getFoods() – возвращает список блюд с их доступностью (рассчитывается на основе остатков продуктов), а также состав в виде строки.
- getProducts() – возвращает список продуктов с текущим количеством и минимальным порогом.
- updateProductQuantity() / addProductQuantity() – изменение остатков.
- checkAndConsume(foodId, quantity) – в транзакции проверяет наличие ингредиентов и списывает их для указанного количества блюда. При нехватке транзакция откатывается.
- returnIngredients() – возвращает ингредиенты при удалении блюда.
- createOrder(), addOrderItem() – создание заказа в БД.

Все публичные методы обрабатывают исключения (классы DatabaseEx, ValidateEx), логируют ошибки и испускают сигнал errorOccurred для уведомления QML.

2.3 Модуль сетевого клиента

Класс TcpClient реализует тот же интерфейс, что и DBWorker, но вместо прямых запросов к БД отправляет команды на сервер. Соединение устанавливается методом connectToServer(). Поток данных построен на JSON-формате: каждая команда – объект с полями command, requestId, data. Ответы приходят асинхронно; для синхронного ожидания используется waitForResponse с таймаутом и локальным циклом обработки событий. При потере соединения генерируется сигнал errorOccurred.

2.4 Модуль сетевого сервера

TcpServer запускает серверный сокет и при подключении клиента создаёт объект ClientConnection, который получает сокет и обрабатывает входящие сообщения. В ClientConnection::onReadyRead() считываются строки, заканчивающиеся символом новой строки, парсится JSON, и в зависимости от команды вызывается соответствующий метод DBWorker. Результат отправляется обратно в виде {"response": true/false, "requestId": N, "data": ...}.

2.5 Модуль логирования

Класс Logs позволяет записывать сообщения в файл и дублировать их в консоль. Поддерживаются уровни: info(), warning(), error(). Методы доступны из QML, поэтому можно логировать события непосредственно из интерфейса.

2.6 Модуль настроек

Класс Settings использует макрос Q_PROPERTY для привязки свойств к QML. Все параметры (хост, порт БД, адрес сервера, режим, путь к логам) сохраняются в системном реестре или конфигурационном файле через QSettings. Методы load() и save() вызываются при запуске и при закрытии диалога настроек.

2.7 Пользовательский интерфейс

В проекте пять QML-файлов:

- LoginPage.qml – экран входа. Содержит поля «Логин», «Пароль», кнопки «Войти» и «Настройки». После успешной авторизации испускает сигнал loginSuccess.
- Main.qml – главное окно с выдвижным меню (Drawer), панелью инструментов (кнопка меню, синхронизация) и StackView для

переключения страниц. В зависимости от роли текущего пользователя меню показывает разные пункты.

- OrderPage.qml – страница оформления заказа. Содержит фильтры (категория, только доступные), список блюд, область текущего заказа (корзина), поля номера столика и количества, кнопки «Добавить», «Удалить», «Оформить заказ». При добавлении вызывается `dbWorker.checkAndConsume`, при успехе блюдо попадает в корзину. Для предупреждения о малых остатках используется `getLowStockWarning` и диалог.
- StockPage.qml – страница управления складом. Список продуктов с подсветкой «ниже порога», поле для ввода количества, кнопки «Добавить», «Убавить», «Установить». Изменения синхронизируются с сервером через методы `addProductQuantity` и `updateProductQuantity`.
- ReportsPage.qml – страница отчётов. Выбор типа отчёта, динамические фильтры (даты, блюдо, столик, официант), кнопка «Сформировать», область вывода результата. Данные получаются методами `getReportRevenue`, `getRevenueByDay`, `getFoodPopularity`, `getReportLowStock`.

3 Интеграция программных модулей

3.1 Подключаемые внешние модули

Проект использует следующие внешние компоненты:

- Qt6::Quick – для компиляции QML-файлов и запуска графического интерфейса.
- Qt6::Sql – для доступа к PostgreSQL.
- Qt6::Network – для работы с сокетами.
- PostgreSQL client library (libpq) – поставляется вместе с PostgreSQL, требуется для работы драйвера QPSQL.

3.2 Интеграция C++ и QML

Для использования методов и полей классов в QML файлах, используются `Q_PROPERTY`, при объявлении полей в заголовочных файлах, и `Q_INVOKABLE`, при объявлении методов и сигналов. Эти макросы позволяют использовать объекты класса, если он зарегистрирован для использования в QML. Для его регистрации, в `main` файле прописывается подобный код:

```
DBWorker *worker = new DBWorker(logger, &app);
```

```
engine.rootContext()->setContextProperty("dbWorker", worker);
```

Таким образом, класс для работы с базой данных регистрируется в программе под именем «dbWorker», в QML файлах его можно использовать именно под таким именем.

3.3 Клиент-серверное взаимодействие

Клиент-серверное взаимодействие построено на TCP-сокетах. Протокол прикладного уровня – JSON. Каждое сообщение завершается символом перевода строки (`\n`), что упрощает разделение пакетов.

Запрос клиента:

```
{"command": "getFoods", "requestId": 1}
```

Ответ сервера:

```
{"response": true, "requestId": 1, "data": {"foods": [...]}}
```

`TcpClient` отправляет запрос, сохраняет `requestId` и ждёт соответствующего ответа (max 60 секунд). Если ответ не получен – выбрасывается исключение `NetworkEx` и генерируется сигнал `errorOccurred`. Сервер (класс `TcpServer`) при подключении нового клиента создаёт объект `ClientConnection`, который асинхронно обрабатывает команды, используя переданный ему экземпляр `DBWorker`. Так достигается параллельность: несколько клиентов обслуживаются одновременно.

4 Описание программного интерфейса (под вопросом)

4.1 Окно авторизации

4.2 Главное окно

4.3 Окно оформления заказа

4.4 Окно склада

4.5 Окно отчётов

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 30389-2013. Услуги общественного питания. Предприятия общественного питания. Классификация и общие требования. — М.: Стандартиформ, 2014. — 16 с.
2. ГОСТ 31985-2013. Услуги общественного питания. Термины и определения. — М.: Стандартиформ, 2014. — 12 с.
3. ГОСТ Р 7.0.96-2016 Система стандартов по информации, библиотечному и издательскому делу. Электронные библиотеки. Основные виды. Структура. Технология формирования. — М.: Стандартиформ, 2016. — 16 с.
4. Инженерия программного обеспечения, 6-е издание. : Пер. с англ. — М. : Издательский дом "Вильямс", 2002 — 624 с. : ил. — Парал. тит. англ.
5. Лебедев Е. Технологии помогают избежать штрафов: об осознанной автоматизации ресторана. — М.: Tochka.com, 2024. — URL: <https://allo.tochka.com/restaurant-automatization>
6. Приказ Минобразования РФ от 19 декабря 2013 г. N 1367 "Об утверждении Порядка организации и осуществления образовательной деятельности по образовательным программам высшего образования – программам бакалавриата, программам специалитета, программам магистратуры". – URL: http://www.consultant.ru/document/cons_doc_LAW_159671/ (дата обращения: 04.08.2016).
7. Стоунз Р., Мэтью Н. PostgreSQL. Основы. – Пер. с англ. – СПб: Символ Плюс, 2002 – 640 с., ил. ISBN 5 93286 043 X
8. ISO 25964-1:2011. Information and documentation – Thesauri and interoperability with other vocabularies – Part 1: Thesauri for information retrieval. – URL: http://www.iso.org/iso/catalogue_detail.htm?csnumber 53657 (дата обращения: 20.10.2016).